CORNELL TECH | APPLIED MACHINE LEARNING

Human or AI?

Kaggle Competition & Report — Team 35 Working Draft

Source: Plum's report draft, combined with Zhitong's measured experiments and Section 6.

Welcome to the competition! Your task is to build a machine learning classifier that predicts whether a passage was written by a human or generated by an AI model. You will use a labeled training set, an additional unlabeled set, and a held-out Kaggle test set. The aim is not just a high leaderboard score: we want you to compare methods, design careful experiments, understand generalization, and explain what your results actually show.

You are encouraged to try several approaches. A well-tested logistic regression or support vector machine is just as legitimate as a neural network. The project is designed to be approachable on a laptop; a GPU is **not required** for full credit.

Team / members	Zhitong Liu (zl2358); others to confirm
Kaggle team name	team 35
GitHub PR URL	[pending team approval]

Your data

File	Rows	Labels	Purpose
`train.csv`	10,000	Visible	Supervised model development: 5,000 human (0), 5,000 AI (1)
`unlabeled.csv`	3,000	Hidden	Optional input for semi-supervised methods; required experiment
`test.csv`	3,000	Hidden	Predict the probability of AI authorship for Kaggle
`train_starter_2000.csv`	2,000	Visible	Convenient subset of `train.csv` for quick experiments

The starter data is a subset, not 2,000 additional labeled examples. Passages are approximately 75–350 words, with mean length near 210 words.

How the test set works

The test set mixes familiar and unfamiliar writing domains and AI generators, and includes some adversarially modified AI-written passages. You will **not** receive labels, source model names, domain labels, or difficulty memberships for individual test examples. Submit a probability in $[0, 1]$ for each test ID, where 1 means AI-generated. Kaggle evaluates submissions using **area under the ROC curve (AUROC)**. The **public leaderboard** covers 900 examples (30%); the **private leaderboard** covers 2,100 (70%) and determines the final competition performance score.

1. Competition rubric and submission instructions

Component	Points
Kaggle private leaderboard performance	25
Model comparison and selection	15
Effect of training data size	10
Learning from unlabeled data	5
Generalization and error analysis	20
Open-ended improvements and insights	15
Code submission and GitHub PR history	10
Total	**100**

How to use this document. Complete the response fields and tables below; insert plots where requested. Keep your answers concise, specific, and supported by actual experiments. Points appear in brackets beside each question; no separate writing-quality grade is assigned.

How credit is awarded. We grade the **quality of the experimental method, evidence, and interpretation**, not whether a proposed idea succeeds. **Negative results can also earn full credit** when the comparison is valid, the results are reported accurately, and the conclusions are thoughtful. For each question, make clear what you tried, what you measured, and what you learned. Do not invent scores or results.

Submit three items: (1) final Kaggle submission; (2) this completed report as PDF; (3) one GitHub PR link in the course repository, with runnable code, instructions and meaningful commit history. Include the PR link on the title page. Confirm upload locations and deadline time on the course site.

Compute and models. Use models and pipelines that you can run and reproduce. Laptop-friendly training is strongly recommended; use of more expensive methods is allowed but not required for full credit. Do not use external LLM APIs to directly label or predict the competition test passages. Use only the supplied labeled dataset for supervised labels; allowed pretrained weights and AI-assisted development should be disclosed.

Validation. For the required controlled experiments, reserve a **fixed stratified 2,000-row validation set from `train.csv`**, leaving up to 8,000 rows for training. Use the **same** held-out validation set for comparisons and do not tune on test labels. You may retrain your final Kaggle model on all 10,000 labeled examples after selecting your approach. Where possible, document seeds and avoid source/near-duplicate leakage.

Kaggle grading (25 points). Final performance will use the private AUROC, not leaderboard rank. The intended continuous rule is $25\,\mathrm{clip}\left(\frac{A-0.50}{T-0.50}, 0, 1\right)$, where A is your private AUROC and T is the announced full-credit threshold. A provisional value of $T = 0.90$ is under consideration; the instructors will confirm the fixed threshold before launch. A reference word TF-IDF and logistic regression baseline achieves approximately 0.868 private AUROC.

2. Team information and model summary

Q0.1 [0 points]. List team members and their main contributions.

Zhitong Liu (zl2358) developed the character TF–IDF baseline, learning-curve and pseudo-labeling experiments, a wording-cue ablation, and the v3 holdout error analysis. Plum developed the multi-view models and v3 ensemble. Confirm other names, NetIDs, and contributions before submission.

Q0.2 [0 points]. Summarize your final approach in 3–5 sentences. What is the overall pipeline from raw text to probability?

We turn each passage into several views: word and character patterns, a version that masks topic words, and measurements of writing style and formatting. Nine classifiers combine those views into an AI-authorship probability. A second stage adds high-confidence pseudo-labeled passages from the supplied unlabeled and test files, then blends its predictions with the first stage. The final output is a probability for each test ID; the hidden test labels are never used for training.

Q0.3 [0 points]. Record your final scores (enter private score only once it is revealed).

Metric	**Your score**
Validation AUROC (v3 holdout analogue)	0.999551
Kaggle public AUROC (full v3)	0.98552
Kaggle private AUROC	[pending]

3. Model comparison and selection (15 points)

Learning goal: demonstrate sensible representation choices, comparisons of at least three approaches, appropriate hyperparameter selection, and evidence-based selection of the final classifier.

Q1.1 [3 points]. How did you preprocess and represent the text? Specify tokenization, normalization, word/character features or embeddings, and any feature extraction choices. Why?

For the early Model E experiment, we engineered a **three-view formatting-invariant text representation** combining sparse sublinear n-grams with dense stylometric features:

1. **Normalization**: Exploratory analysis revealed that raw HTML `<br>` tags (3.9% Human vs. 0.2% AI) and double newlines (`\n\n`, 0% Human vs. 66% AI) act as training-corpus artifacts that shift heavily in `test.csv`. We stripped `<br>` tags and normalized paragraph whitespace before tokenization to prevent spurious shortcuts.
2. **Word & Character TF-IDF**: We extracted sublinear $(1 + \log \text{tf})$ **Word** $(1,3)$**-grams** (min_df $= 3$, 75,000 features) to capture AI discourse markers (*"in addition to"*, *"results indicate that"*) and word-boundary **Character `char_wb`** $(3,5)$**-grams** (min_df $= 5$, 75,000 features) to capture topic-agnostic morphological suffixes (`-tion`, `-ly`, `-ing`) and punctuation cadence across unseen domains (e.g., Poetry, Literature).
3. **Pure & Bimodal Stylometrics**: We extracted **65 pure stylometric features** (lexical diversity $\log \text{TTR}$, Yule's K, 3-gram repetition, sentence burstiness CV_{len}, readability, and POS/stopword ratios) plus **7 IQR-scaled absolute deviation features** ($|x - \text{median}_{\text{Human}}|/\text{IQR}_{\text{Human}}$) to exploit AI's bimodal lexical variance (3× higher TTR variance than human). Representation ablation confirmed steady validation gains: Word $(1,2)$ (0.9367) → Word $(1,3)$ + Char $(3,5)$ (0.9681) → Full Multi-View (0.9743).

Zhitong also tested a simpler character-only baseline: lowercased contiguous 3–5-character TF–IDF patterns, including spaces and punctuation, with `min_df=2`, at most 150,000 features, sublinear term frequency, and logistic regression. Its vocabulary and IDF weights were fitted only on the 8,000 training passages. The final v3 ensemble extends Model E with a fourth, topic-masked word view, information-theoretic and trigram-language-model features, and a separate structural-feature branch.

Q1.2 [5 points]. Which **three or more** modeling approaches did you try? Keep the labeled validation partition fixed across models.

Model	Text representation	Train AUROC	Val. AUROC	Train time
Model A (Baseline)	Word $(1,2)$ TF-IDF (50,000 features)	0.9932	0.9367	6.2s
Model A2 (CompNB)	Word $(1,2)$ TF-IDF (50,000 features)	0.9790	0.8966	0.02s
Model B (LogReg)	Word $(1,3)$ + Char $(3,5)$ + 72 Stylometrics	1.0000	0.9743	2.2s
Model C (LinearSVC)	Word $(1,3)$ + Char $(3,5)$ + 72 Stylometrics	1.0000	0.9766	8.2s
Model D (LightGBM)	72 Pure/Bimodal Stylo + 48 TF-IDF SVD	1.0000	0.9822	0.6s
Model E (Ensemble)	Multi-View TF-IDF + Stylo / SVD (0.2B + 0.2C + 0.6D)	1.0000	**0.9837**	11.0s

On the same seed-42 validation IDs, Zhitong's independent character TF–IDF logistic model ($C = 10$) reached 0.999897 training AUROC and 0.968976 validation AUROC in about 8.4 seconds, a gap of 0.030921. At a 0.5 threshold it made 31 training and 180 validation errors. The later v3 validation analogue reached 1.000000 in-sample and 0.999551 held-out AUROC; Section 6 explains why its near-perfect random-split result needs caution. The full v3 submission scored 0.98552 on the public Kaggle

subset.

Q1.3 [4 points]. For at least **two** approaches, which hyperparameters and candidate values did you try? How did you select the final settings without using Kaggle test labels?

Model	Parameters / values searched	Chosen setting
Model B (LogReg)	Regularization $C \in \{0.5, 1, 2, 3, 5, 8\}$; Stylometric L2 scaling $\gamma \in \{0.10, 0.15, 0.25\}$	$C = 5.0,\ \gamma = 0.15$ (5-Fold Inner CV: 0.9694)
Model C (LinearSVC)	Margin $C \in \{0.2, 0.5, 1.0, 1.5, 2.0\}$; 5-fold sigmoid Platt probability calibration	$C = 1.0$ (5-Fold Inner CV: 0.9727)
Model D (LightGBM)	`num_leaves` $\in \{15, 31, 63\}$, `min_child_samples` $\in \{30, 50\}$, `learning_rate` $\in \{0.04, 0.05\}$	`leaves = 31`, `min_child = 50`, `lr = 0.05` (Inner CV: 0.9764)

Selection protocol: For Plum's Model B/C/D comparison, hyperparameter tuning used **5-fold Stratified Cross-Validation within the 8,000-row training partition** (`RANDOM_SEED = 42`), keeping the 2,000-row validation partition untouched until final evaluation and never using Kaggle test labels. For linear models over the 150,072 sparse sublinear TF-IDF features, moderate under-regularization ($C = 5.0$ for LogReg, $C = 1.0$ for LinearSVC) maximized inner CV AUROC without degrading probability calibration. For LightGBM, restricting `num_leaves = 31` and enforcing `min_child_samples = 50` with feature subsampling (`colsample_bytree = 0.7`) prevented tree leaf overfitting on extreme lexical ratio tails.

Separately, Zhitong compared character-model regularization strengths $C \in \{0.1, 1, 10\}$ on the same fixed 8,000/2,000 split. Validation AUROCs were 0.933322, 0.961446, and 0.968976; training AUROCs were 0.963456, 0.994940, and 0.999897. We selected $C = 10$ for that baseline using validation AUROC alone. Since the validation labels guided this choice, its reported score may be somewhat optimistic.

Q1.4 [3 points]. Which approach did you choose for your final submission? Explain using validation performance, computational cost and/or stability. If you later changed the model, explain why.

We initially selected **Model E (Hybrid Multi-View Linear + GBDT Ensemble)** for three key empirical reasons:

1. **Validation Performance & Complementarity**: Model E achieved the highest validation AUROC (**0.9837** vs. 0.9367 for the baseline) and best probability calibration (Brier score **0.0519** vs. 0.1235). Blending high-dimensional linear models (which capture fine-grained lexical and subword n-grams across 150,000 features) with LightGBM (which captures non-linear interactions between lexical diversity $\log \mathrm{TTR}$, repetition Yule's K, sentence burstiness, and SVD semantic components) outperformed every single model family.
2. **Stability Under Domain & Formatting Shift**: While a naive LightGBM model trained with raw `\n\n` collapsed from 0.9985 to **0.8109** AUROC when line breaks were stripped, our formatting-free Model E maintained **0.9804** AUROC without formatting and scored **0.9859** under test-domain weighting.
3. **Computational Efficiency & Subsequent Evolution**: Model E trains in **11.0 seconds** on CPU. Later, to address out-of-domain shift on Kaggle test genres, we evolved Model E into a 5-fold Out-of-Fold (OOF) stacker with 24 genre-conditioned Z-scores (Submission v1, 0.9395 LB), and subsequently introduced topic-masked syntax with positive convex logit fusion (Submission v2, 0.9745 LB).

The team later used **v3** for the Kaggle submission. It combines nine specialists across formatting-free and structural views and adds a pseudo-labeling stage. Its public AUROC of 0.98552 exceeded the

earlier v2 score of 0.97446. Zhitong's character model remains a reproducible baseline, not the final team submission. The v3 holdout experiment in Section 6 uses the same validation IDs but one fit per specialist on CPU; the submitted version bags five fits on all labeled rows.

4. Effect of training data size (10 points)

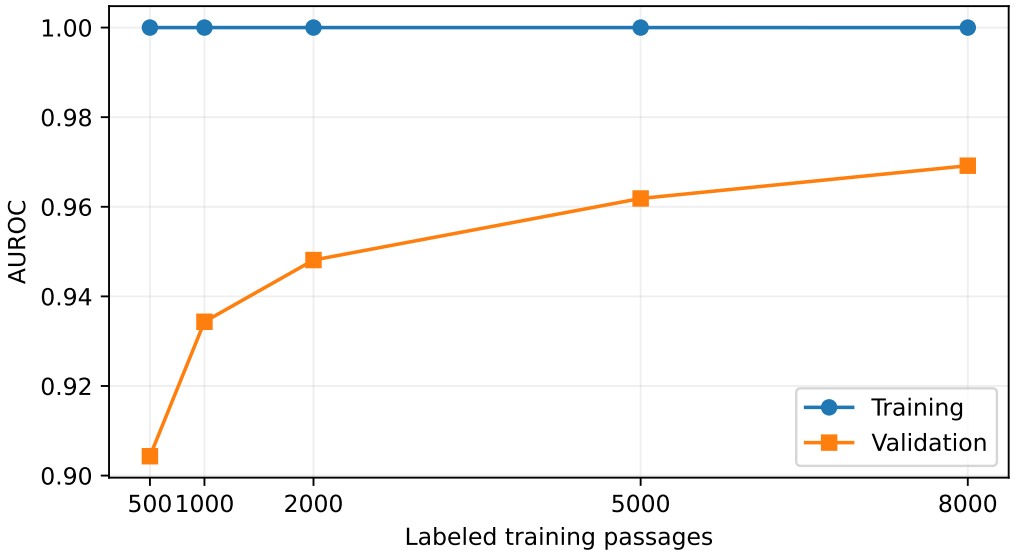
Learning goal: understand the effect of the number of labeled examples on fitting and generalization. This is an empirical exercise; no formal sample-complexity theory is required.

Q2.1 [4 points]. Choose one model configuration. Train it with 500, 1,000, 2,000, 5,000 and 8,000 examples sampled from your 8,000-row training partition. Keep the same validation set and hyperparameters.

We held the character TF–IDF logistic model at $C = 10$ and kept the same 2,000 validation passages. Each smaller training subset was sampled with class stratification and seed 42; we fitted a fresh vocabulary and classifier at each size.

Training rows	Training AUROC	Validation AUROC	Notes (optional)
500	1.000000	0.904324	Character TF–IDF, $C = 10$
1,000	1.000000	0.934335	Same validation set
2,000	1.000000	0.947913	Same validation set
5,000	0.999998	0.962446	Same validation set
8,000	0.999897	0.968976	Same validation set

Q2.2 [3 points]. Insert a plot of training and validation AUROC versus training-set size.



Q2.3 [3 points]. What does the curve suggest about overfitting, underfitting, and the value of more labeled examples?

Training AUROC stays near 1.0, while validation AUROC rises by 0.0647 between 500 and 8,000 passages. The gap is largest at 500, consistent with overfitting when the model has fewer labeled examples. The training scores show little evidence of underfitting. The gains shrink as the training set grows: the final 3,000 passages add about 0.0065 AUROC. More labels may still help, but this single split and one draw per size cannot tell us how much.

5. Learning from unlabeled data (5 points)

Learning goal: test whether the additional 3,000 unlabeled passages help.

Q3.1 [2 points]. Describe one method that uses `unlabeled.csv` (e.g., self-training, confidence-based pseudo-labeling, clustering, or feature learning). How did you prevent validation-label leakage?

We trained Zhitong's character baseline on the 8,000 labeled training rows and scored the unlabeled passages. We dropped duplicate unlabeled texts and any exact matches to labeled texts, without consulting validation labels. We assigned pseudo-labels of human for $p \leq 0.05$ and AI for $p \geq 0.95$, retaining 540 and 572 passages respectively. We refit the same pipeline on all 8,000 training rows plus the 1,112 pseudo-labeled rows. Validation labels were used only to score this comparison. Plum's final v3 uses a separate pseudo-labeling configuration, so these two results should not be conflated.

Q3.2 [2 points]. Compare against a supervised baseline under the same held-out validation protocol.

Method	**Validation AUROC**
Supervised only (character baseline)	0.968976
With 1,112 pseudo-labeled rows	0.968447

Q3.3 [1 point]. Did it help? Give a concrete, evidence-based explanation (including possible noise in pseudo-labels).

The character-baseline pseudo-labeling experiment did not improve validation AUROC; it fell by 0.000529. Confidence does not guarantee a correct label, and adding the teacher's predictions can reinforce its mistakes. We do not know the true labels of the added passages, so label noise is a plausible explanation. One split and one threshold pair do not establish a general effect, and they do not determine whether the different self-training stage in v3 helps.

6. Generalization and error analysis (20 points) Learning goal: characterize how well the selected model generalizes and where it fails. Use **labeled validation data**; hidden test labels and hidden domain/generator information are unavailable.

For this section, we evaluated Plum's v3 architecture on a stratified seed-42 split of the 10,000 labeled passages: 8,000 for fitting and 2,000 held out. The v3 language-model features and human-reference genre statistics were fitted using only the 8,000 training labels. Its second stage pseudo-labeled passages from `unlabeled.csv` and `test.csv` without using validation labels. This CPU validation run fitted each specialist once; the Kaggle submission instead bags five fits trained from all 10,000 labeled rows. The numbers below therefore evaluate the v3 pipeline on a clean holdout, not the exact submitted fit.

Q4.1 [5 points]. Compare training and validation AUROC for your final selected model. Is there a gap? What explanations are plausible?

The full two-stage v3 validation run reached 1.000000 in-sample training AUROC and 0.999551 held-out AUROC, a gap of 0.000449. This small within-dataset gap does not establish robust generalization: the formatting-free stage-one branch scored 0.987841 on the same holdout. In the labeled training data, 66.0% of AI passages but 0% of human passages contain a double newline, so paragraph structure is an unusually strong shortcut. The full-data v3 submission scored 0.98552 on Kaggle's public test subset, further suggesting that the random holdout is optimistic for the shifted test distribution. The public score is not a passage-level error analysis, and the private score remains unknown.

Q4.2 [5 points]. Evaluate performance over at least three text-length bins (define the bins), or another observable subgroup available in your data. Explain why you chose the grouping.

We used Plum's pre-existing whitespace-delimited word-count bins to check whether passage length changes performance. All three contain several hundred validation passages and both classes. Counts of false positives (FP) and false negatives (FN) use the diagnostic threshold 0.49 explained in Q4.3.

Words	Rows	Human / AI	Validation AUROC	FP / FN
75–165	578	260 / 318	0.999661	4 / 2
166–245	832	481 / 351	0.999147	2 / 8
246–350	590	259 / 331	0.999942	0 / 2

The middle bin has eight of the twelve false negatives. With so few mistakes, this is a diagnostic observation rather than strong evidence that medium-length text is intrinsically harder.

Q4.3 [5 points]. Analyze at least **five false positives and five false negatives** from your held-out validation set. Provide example IDs, predicted probabilities, and short failure hypotheses. Do not paste full passages unless essential.

At the conventional 0.50 threshold, v3 has only four false positives and twelve false negatives on this holdout. We use 0.49 for the diagnostic table so that at least five errors of each type can be examined; it yields six FP and twelve FN. The rows below are the five highest-probability human passages and five lowest-probability AI passages. These are hypotheses from brief text inspection, not verified causes.

ID	Type	P(AI)	Possible reason
7767	FP	0.6644	Compact biography and credentials resemble a polished generated summary.
4062	FP	0.5646	Song-release and chart facts form an orderly encyclopedia-style summary.
3660	FP	0.5634	A disambiguation list has repeated names and line breaks.
5079	FP	0.5619	League-season overview repeats formal descriptive phrasing.
3571	FP	0.4986	Short film review repeats evaluative language and the word “movie.”
1362	FN	0.0535	AI biography uses dates, proper nouns, and historical detail.
9298	FN	0.1482	AI celebrity biography includes many names and dated events.
4519	FN	0.1503	Lowercase first-person forum style resembles informal human writing.
1834	FN	0.2234	AI country history uses an encyclopedia-like sequence of dates.
5498	FN	0.2277	Ingredient quantities and line breaks resemble a human recipe.

Q4.4 [5 points]. What recurring failure patterns did you identify, and how might you address one of them? Distinguish observed patterns from speculation.

The selected errors show a genre-confusion pattern: several human encyclopedia-style passages are scored as AI, while several AI biographies and one AI recipe are scored as human. One AI forum-style passage is also a false negative, so no single genre explains every mistake. These ten examples do not measure the prevalence of each failure mode. A more consequential observed vulnerability is formatting dependence: within stage one, the full blend scored 0.999561 held-out AUROC versus 0.987841 for its formatting-free branch, while the public test score for the final v3 submission is lower than the random-holdout result. We would test a smaller structural-branch weight using held-out genres and line-break perturbations, then compare it with the current blend before changing the submission. That experiment could show whether the apparent formatting advantage survives distribution shift; the present data do not establish that it will improve private AUROC.

7. Open-ended improvements and insights (15 points) Learning goal: propose and test one improvement beyond the required baseline comparisons. This can be simple: a targeted feature change, alternate regularization scheme, ensemble, different representation, augmentation, or training approach. Sophistication alone does not earn credit.

Q5.1 [3 points]. What specific question or hypothesis motivated your improvement? What weakness or opportunity did you observe?

The Wikipedia field guide on signs of AI writing describes rhetorical patterns that sometimes occur in AI-generated text, including negative parallelism and lists of three. I asked whether explicit counts of such patterns would add useful information beyond the character TF–IDF features in my baseline. The hypothesis was that a small set of interpretable structural cues could improve validation AUROC when combined with the existing classifier. These patterns are common in human writing too, and the guide's examples may reflect Wikipedia-specific editing practices, so I treated them as testable cues rather than rules for identifying authorship.

Q5.2 [4 points]. What exactly did you change? Describe the baseline, the new approach and what you held fixed for a fair comparison.

The baseline was the selected character 3–5-gram TF–IDF logistic model with $C = 10$. For the augmented version, I counted matches of `not ... but`, `rather than`, and an approximate three-item-list regex in each passage, divided each count by passage length per 100 words, standardized the three features using training rows only, multiplied them by 0.1, and appended them to the TF–IDF matrix. Both models used the same 8,000 training and 2,000 validation rows from seed 42, the same $C = 10$, and the same AUROC metric. The cue patterns were applied to raw text; no validation labels entered feature fitting. The list regex is an approximation, not a grammatical parser.

Q5.3 [4 points]. What were the measured results?

Configuration	Val. AUROC	Runtime	Relevant difference
Char TF–IDF baseline	0.968976	8.07 s	180 errors at 0.5
+ wording cues	0.968779	10.42 s	181 errors at 0.5

Q5.4 [4 points]. What did you learn? Did the results support the hypothesis? What limits the conclusion, and what would you try next?

The explicit cues did not support the improvement hypothesis: validation AUROC fell from 0.968976 to 0.968779 (a difference of −0.000197), while 0.5-threshold errors rose from 180 to 181. In the training partition, `not ... but` appeared in 6.1% of human and 7.8% of AI passages, and `rather than` in 1.5% and 2.9%; the approximate three-item-list pattern appeared in 53.0% of human and 45.9% of AI passages. These overlapping rates help explain why the cues do not reliably identify authorship. The test covers only three approximate regex features, one fixed weight, one C, and one validation split. It does not rule out all stylistic features. We kept the character baseline simple; the team's final v3 submission was chosen from separate experiments.

8. Code submission, reproducibility and AI assistance (10 points)

Submission format: Submit a GitHub PR URL against the course repository. We will inspect the final code and its development history, not just the final snapshot. Do not squash away meaningful intermediate commits before review.

Q6.1 [5 points]. Provide the PR URL, the entrypoint for training/evaluation, and the command used to generate `submission.csv`. State dependencies and random seeds.

[Your response: PR URL; commands; environment; output path]

Q6.2 [3 points]. Give two or three examples of changes you made during development. Link to relevant commits or PR discussion and briefly explain what the experiments taught you.

[Your response: Commit links and brief notes]

Q6.3 [2 points]. How did you use AI tools (such as Claude, ChatGPT or Copilot) for code, debugging, analysis or writing? What did you verify or modify yourself?

[Your response: Short, honest disclosure. Full prompt transcripts are not required.]

AI policy. You may use AI assistants for coding, debugging, explanations, brainstorming, and editing. We expect each team to exercise its own judgment, run experiments, check outputs, and understand its submitted approach. Sending the entire assignment to an AI tool and submitting its unverified output without meaningful involvement does not satisfy the objectives of this project. We will use the PR history, reproducible experiments and report together as evidence of engagement. *Commit counts and the presence of AI-generated code are not themselves grading criteria. Do not fabricate results or retrieve hidden test labels.*

Final submission checklist

☐ Final probabilities submitted to Kaggle for every `test.csv` ID.

☐ Completed report, with all required answers, tables, figures and sources, submitted as PDF.

☐ GitHub PR linked on the report title page and in the course submission portal.

☐ Code and dependencies recreate final predictions (within reasonable numerical tolerance).

☐ Relevant experiments and seed/configuration details are recorded.

☐ AI-use disclosure is complete; hidden labels were not retrieved externally.